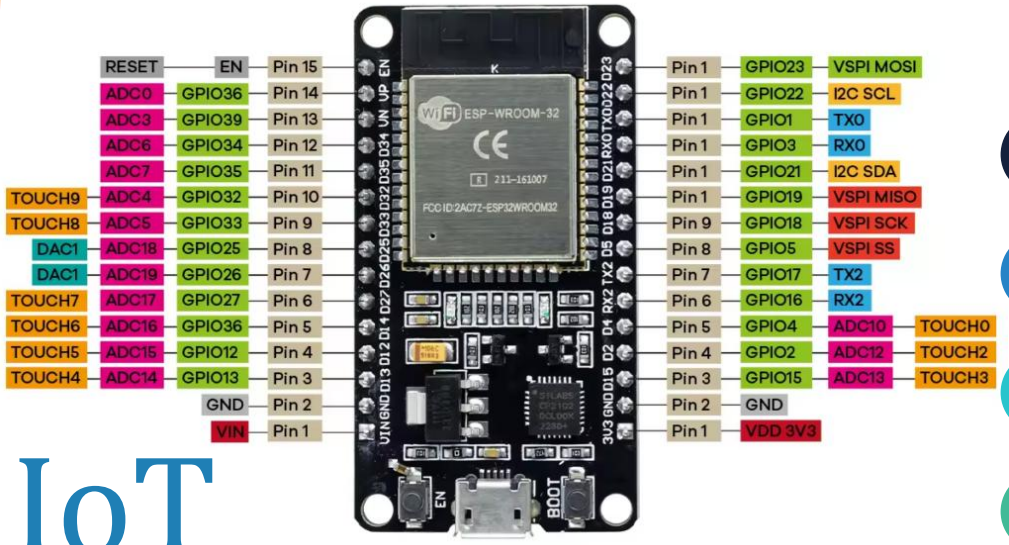
Mission Raftaar

ESP32 & IoT

Hands-on Training for Educators



Dual-core 240 MHz

Wi-Fi + Bluetooth LE

34 GPIO pins

I2C | SPI | UART

EMBEDDED SYSTEMS | ARDUINO PROGRAMMING | WI-FI & MQTT | ESP32-CAM | TINYML

NIELIT CHANDIGARH / ROPAR



Module 1

Embedded Systems & IoT

FOUNDATIONS BEFORE THE FIRST LINE OF CODE

EMBEDDED SYSTEMS | MICROCONTROLLER VS MICROPROCESSOR | IOT ARCHITECTURE | APPLICATIONS

Learning Outcomes and Training Roadmap

UNDERSTAND

- ESP32 architecture, memory and peripherals
- Microcontroller vs microprocessor
- IoT layers and data flow
- Edge AI and TinyML basics

BUILD

- Program the ESP32 in Arduino C/C++
- Use GPIO, PWM, ADC, DAC, I2C, SPI, UART
- Interface sensors, displays, servos
- Web server, MQTT and cloud dashboards

TEACH

- Design practicals and live demos
- Run interdisciplinary mini-projects
- Assess with MCQs, rubrics, challenges
- Troubleshoot common student errors

Training roadmap | suggested 2-3 day programme, about 16 contact hours

M1	M2	M3	M4	M5	M6	M7
Embedded & IoT basics	ESP32 hardware	Arduino programming	Sensors & actuators	Wi-Fi, web, MQTT	Camera, AI, apps	Teaching & projects

Each module: concept > circuit > code > expected output > classroom activity > knowledge check

What Are Embedded Systems?

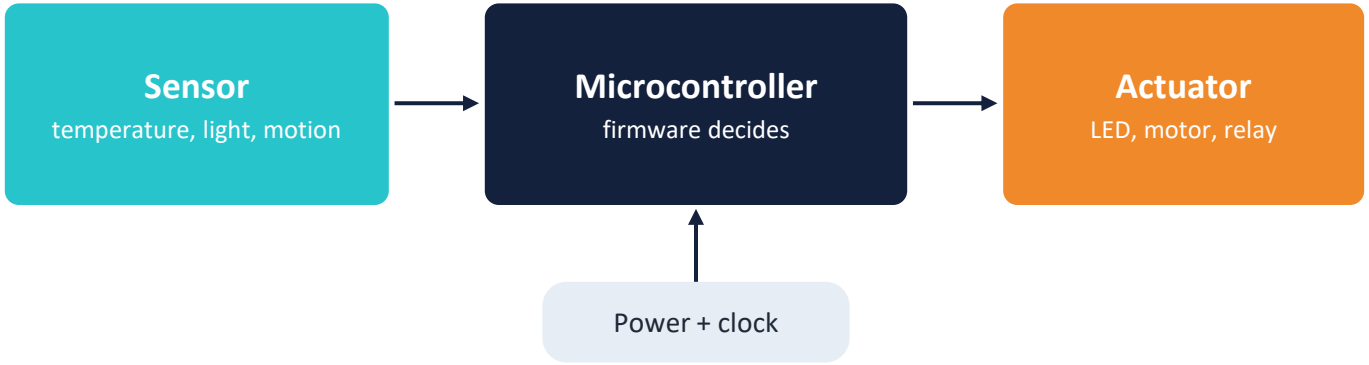
WHAT IS IT?

- A computer built into a product to do one dedicated job
- Hardware + firmware, usually real-time and low-power

WHY DO WE NEED IT?

- Automates sensing and control cheaply and reliably
- Small, always-on, runs for years on little energy
- Billions of devices: phones, cars, meters, toys

How does it work? Sense > Decide > Act



Everyday examples

Washing machine	Car ABS	Smart meter
Pacemaker	Drones	Smart watch

TIP

Analogy: an embedded system is like a thermostat with a brain: it keeps sensing the room, compares with a target, and switches the heater on or off, forever.

Ask: name three embedded devices in this room and what each one senses.

Microcontroller vs Microprocessor

Feature	Microcontroller (e.g. ESP32)	Microprocessor (e.g. Core i5, Cortex-A)
What is inside	CPU + RAM + flash + I/O peripherals on ONE chip	CPU only; RAM, storage, I/O are external chips
Typical clock	16 MHz - 240 MHz	1 GHz - 5 GHz
Memory	KB to a few MB (ESP32: 520 KB SRAM)	GB of external RAM
Software	Firmware, bare-metal or RTOS	Full OS: Linux, Windows, Android
Power	Milliwatts; deep sleep in microamps	Watts to tens of watts
Boot time	Instant (milliseconds)	Seconds to minutes
Best for	Dedicated control: sensors, motors, IoT nodes	General computing, GUI, heavy AI, servers
Cost	Rs 300 - 800 for an ESP32 board	Higher; needs a motherboard

TIP

Analogy: a microprocessor is a powerful chef who needs a whole kitchen (RAM, storage, I/O). A microcontroller is a food truck: small, self-contained, does one menu very well.

CHECK

Is a Raspberry Pi a microcontroller? Its SoC runs Linux, so it is a microprocessor-class board. (Pi Pico is a microcontroller.)

Introduction to the Internet of Things

WHAT IS IT?

A network of physical objects with sensors, software and connectivity that collect and exchange data over the Internet.

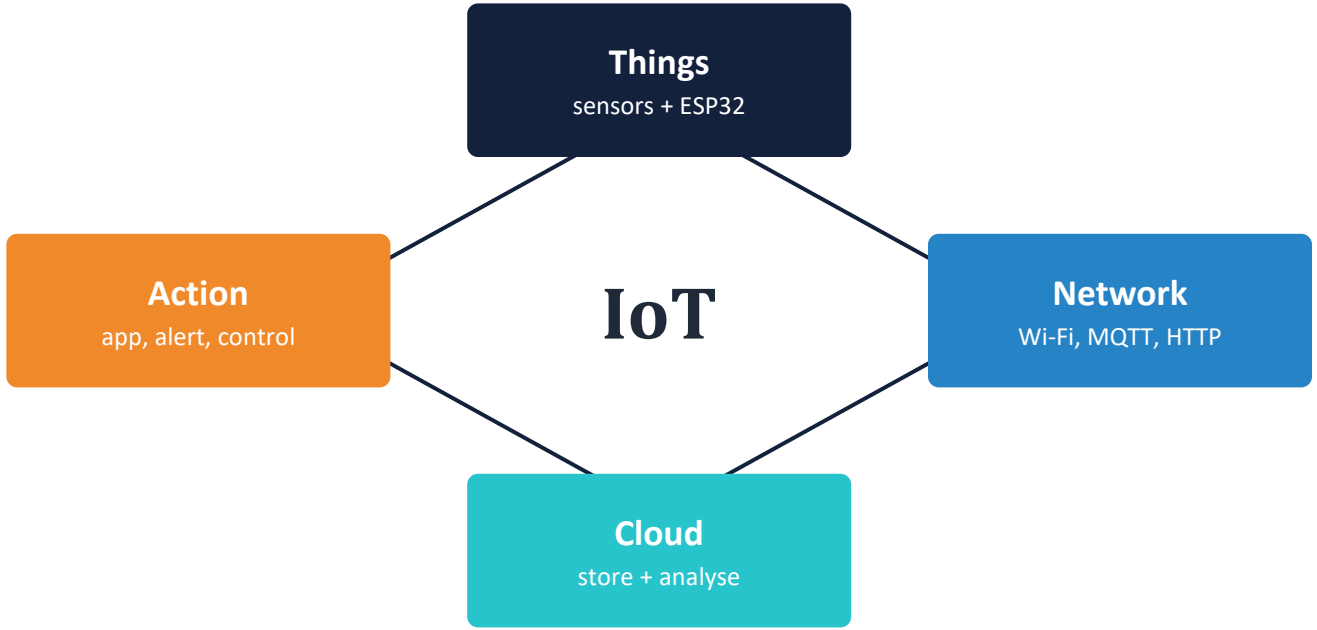
WHY IS IT REQUIRED?

- Remote monitoring without a person on site
- Data-driven decisions and automation
- Early fault detection saves cost and lives

HOW DOES IT WORK?

Device senses > sends over Wi-Fi/LTE/LoRa > cloud stores and analyses > user sees dashboard or app triggers an action back.

IoT data loop



TIP

Scale: tens of billions of connected devices worldwide. ESP32 is a popular, low-cost entry point for learning IoT end to end.

IoT Architecture: Four Layers

4 Application layer	**Dashboards, mobile apps, alerts, analytics, automation rules** e.g. Grafana, ThingSpeak, Node-RED, Blynk
3 Processing layer	**Storage, rules, cloud and edge analytics, machine learning** e.g. cloud server, MQTT broker, edge AI on ESP32
2 Network layer	**Moves data reliably and securely between device and cloud** e.g. Wi-Fi, Bluetooth LE, LoRa, LTE; MQTT, HTTP
1 Perception layer	**Sensors read the world; actuators change it** e.g. DHT22, PIR, LDR, relay, servo + ESP32

Data flows UP

Commands flow DOWN

ESP32 spans layers 1 and 2

and can run layer 3 logic at the edge

Activity: map a smart street-light project onto the four layers (which device sits in which layer?)

Real-World IoT Applications

E **Education**

Smart attendance, lab monitoring, remote experiments

A **Agriculture**

Soil moisture, auto irrigation, greenhouse climate

H **Healthcare**

Wearable vitals, remote patient monitoring, cold chain

I **Industry (IIoT)**

Predictive maintenance, energy meters, asset tracking

S **Smart cities**

Street lights, parking, waste bins, air quality

S **Smart home**

Lighting, security, energy saving, voice control

TEACHING TIP

Classroom idea: every student group picks one domain and sketches sensors, ESP32 role, cloud and user action on one page.



Module 2

ESP32 Hardware & Architecture

KNOW THE CHIP BEFORE YOU WIRE IT

ESP32 FAMILY | BLOCK DIAGRAM | CPU AND MEMORY | GPIO PINOUT | ADC, DAC, PWM | UART, SPI, I2C | WI-FI AND BLUETOOTH

Introduction to the ESP32

WHAT IS IT?

A low-cost, low-power system-on-chip from Espressif with a dual-core CPU, Wi-Fi and Bluetooth, designed for IoT.

WHY USE IT IN TEACHING?

- Wi-Fi + Bluetooth built in: IoT in one board
- Programmable with Arduino IDE, MicroPython, ESP-IDF
- Rich peripherals; costs about the price of a textbook
- Huge community, libraries and tutorials

HOW DOES IT WORK?

Firmware in flash runs on the Xtensa cores; peripherals are configured through registers and driven by the Arduino API.

240 MHz dual-core Xtensa LX6	**520 KB** SRAM on chip	**34** GPIO pins
2.4 GHz Wi-Fi 802.11 b/g/n	**BLE 4.2** plus Bluetooth Classic	**3.3 V** logic level

TIP

Teaching tip: hold up the board and name the three things students can see: the silver module (CPU + radio + flash), the USB-UART chip, and the pin headers.

REAL-WORLD USE

Smart plugs, sensor nodes, BLE beacons, cameras, wearables and classroom lab kits.

ESP32 Family and Development Boards

Chip	CPU	Wireless	Highlights	Use it for
ESP32	Xtensa LX6, 2 cores, 240 MHz	Wi-Fi 4 + BT Classic + BLE 4.2	Most boards and tutorials; DAC, touch	This course
ESP32-S2	Xtensa LX7, 1 core	Wi-Fi 4 only	Native USB, more GPIO	Wi-Fi only, USB devices
ESP32-S3	Xtensa LX7, 2 cores	Wi-Fi 4 + BLE 5	AI vector instructions, USB, camera	Edge AI, cameras
ESP32-C3	RISC-V, 1 core, 160 MHz	Wi-Fi 4 + BLE 5	Low cost, open RISC-V	Cheap IoT nodes
ESP32-C6	RISC-V, 160 MHz	Wi-Fi 6 + BLE 5 + 802.15.4	Thread / Zigbee / Matter	Smart-home
ESP32-H2	RISC-V, 96 MHz	BLE 5 + 802.15.4 (no Wi-Fi)	Very low power	Thread / Zigbee nodes

Common development boards

ESP32 DevKit V1 (30/38-pin)
Our lab board: ESP32-WROOM-32, USB-UART, breadboard friendly

ESP32-CAM (AI-Thinker)
OV2640 camera + microSD, needs FTDI programmer

ESP32-S3-DevKitC
AI-capable, native USB, camera support

LILYGO / Heltec boards
Built-in OLED, LoRa or TFT display variants

Pick for class: original ESP32 DevKit V1. Every pin number in this deck refers to it; check your board silkscreen first.

ESP32 Internal Architecture

ESP32 SoC (single chip)

Dual-core CPU
Xtensa LX6: PRO_CPU (core 0) + APP_CPU (core 1), up to 240 MHz, FreeRTOS

Memory
ROM 448 KB | SRAM 520 KB, RTC memory 16 KB (keeps data in deep sleep)

Radio
Wi-Fi 802.11 b/g/n (2.4 GHz), Bluetooth Classic + BLE 4.2

Internal bus (AHB / APB)

GPIO x34	ADC 12-bit x18	DAC 8-bit x2	Touch x10	PWM (LEDC) x16	I2C x2
SPI x4	UART x3	I2S x2	CAN (TWAI)	SDIO	Ethernet MAC

Support blocks

Security
AES, SHA, RSA, RNG: secure boot, flash encryption

Clock & power
40 MHz crystal, PLL: sleep modes, ~10 uA deep sleep

ULP co-processor
runs while the CPUs sleep, wakes on sensor threshold

Off-chip (on module)
SPI flash 4 MB (firmware), optional PSRAM, antenna

CPU, Memory and Processing Capability

Where your program lives

Flash (4 MB typical)
program code, constants, SPIFFS / LittleFS files

SRAM (520 KB)
variables, stack, heap; Wi-Fi buffers use ~40-60 KB

RTC memory (16 KB)
survives deep sleep (RTC_DATA_ATTR variables)

ROM (448 KB)
boot loader and core functions (read only)

PROCESSING

- Two 32-bit cores; Arduino code runs on core 1 by default
- Up to ~600 DMIPS; hardware floating point unit
- Run tasks on both cores with FreeRTOS xTaskCreatePinnedToCore()

LIMITS TO REMEMBER

- SRAM is small: avoid huge String concatenations
- Do not block loop() with long delay() when serving Wi-Fi
- Watchdog resets a program that never yields

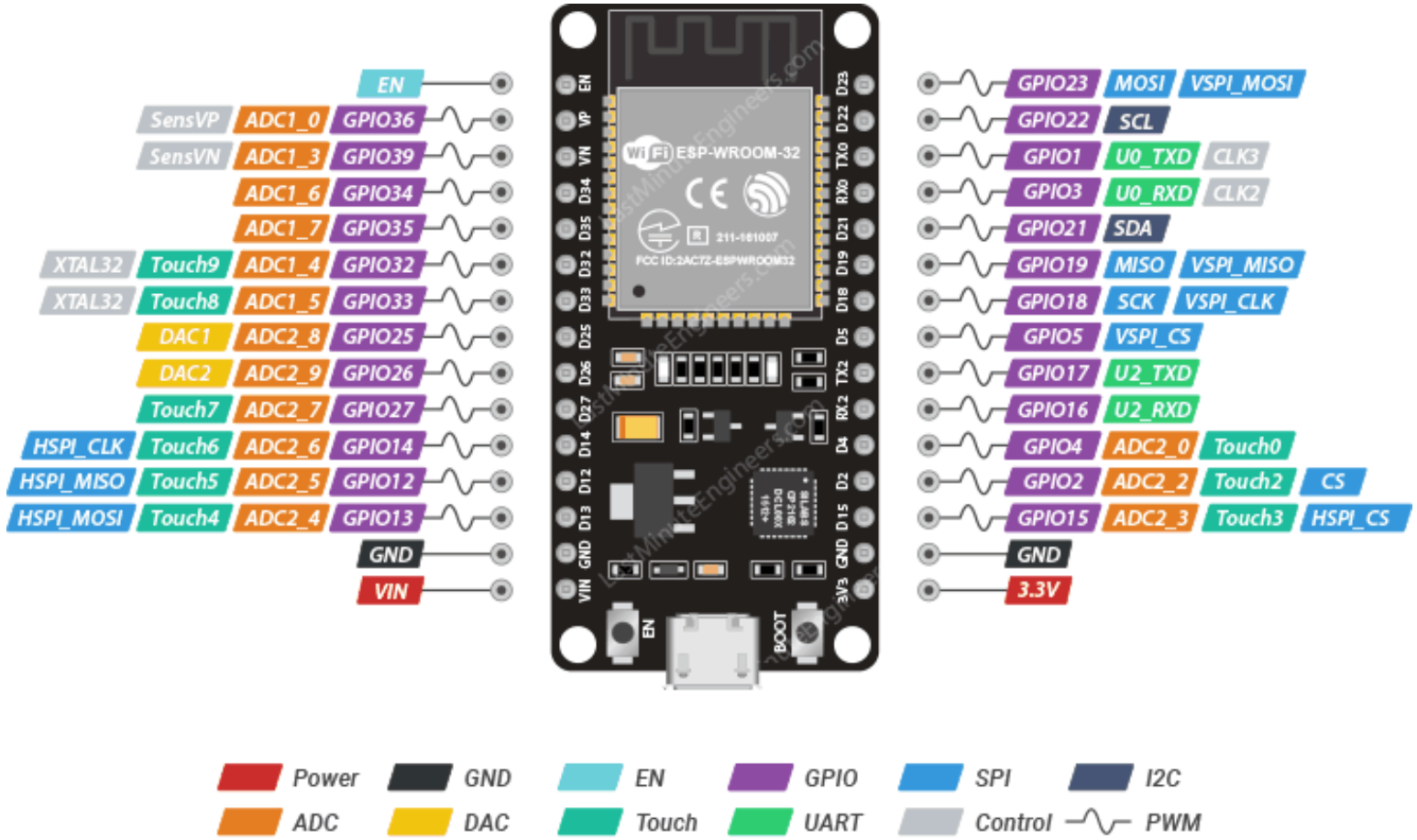
TRY IT: PRINT CHIP INFO

```
Serial.println(ESP.getChipModel());
Serial.println(ESP.getFreeHeap());
Serial.println(ESP.getFlashChipSize());
```

TIP

Activity: run the three lines above on every board. Compare free heap before and after connecting Wi-Fi. Why did it drop?

GPIO Pinout and Pin Functions



ESP32 Dev. Board Pinout

Last Minute ENGINEERS.com

ADC, DAC, PWM, Touch and Interrupts

ADC: ANALOG IN

- 12-bit (0-4095), about 0-3.3 V
- Use ADC1 pins (GPIO32-39) when Wi-Fi is on
- ADC2 pins are blocked by Wi-Fi

```
analogRead(34);
```

DAC: ANALOG OUT

- Two true 8-bit DACs: GPIO25, GPIO26
- Output 0-3.3 V in 256 steps
- Waveforms, audio, reference voltage

```
dacWrite(25, 128);  // 1.65 V
```

PWM (LEDC)

- 16 channels on any output pin
- Frequency and resolution are set by you
- Dim LEDs, drive servos, control motor speed

```
ledcAttach(18, 5000, 8);
ledcWrite(18, 128);
```

TOUCH SENSING

- 10 capacitive pins: T0=GPIO4, T2=2, T3=15, T4=13, T5=12, T6=14, T7=27, T8=33, T9=32
- Reading falls when touched

```
touchRead(T0);
```

INTERRUPTS

- Nearly every GPIO can trigger one
- Event-driven: no polling, instant reaction
- ISR must be short; mark with IRAM_ATTR

```
attachInterrupt(digitalPinToInterrupt(4),
  isr, FALLING);
```

REMEMBER

- ESP32 ADC is non-linear near 0 V and 3.3 V; calibrate for accuracy
- GPIO34-39: input only, no pull-ups
- Max 3.3 V on every pin
- Core 3.x: ledcAttach(pin, freq, bits); core 2.x used ledcSetup + ledcAttachPin

Communication: UART, SPI and I2C

	UART	SPI	I2C
Wires	2 (TX, RX) + GND	4 (SCK, MOSI, MISO, CS)	2 (SDA, SCL) + pull-ups
Style	Asynchronous, point to point	Synchronous, full duplex	Synchronous, shared bus, addressed
Speed	9600 - 115200 baud typical	Up to 80 MHz; fastest	100 kHz / 400 kHz
Devices	GPS, GSM, serial PC link	TFT, SD card, ADC, LoRa	OLED, LCD, MPU6050, RTC
ESP32 default pins	UART0: 1/3 | UART2: 17/16	VSPI: 18, 19, 23, CS 5	SDA 21, SCL 22

UART2 in code

```
Serial2.begin(9600, SERIAL_8N1,
  16, 17);  // RX, TX
```

SPI (VSPI) in code

```
SPI.begin(18, 19, 23, 5);
// SCK, MISO, MOSI, SS
```

I2C in code

```
Wire.begin(21, 22);
// SDA, SCL (any pins work)
```

TIP

Analogy: UART = a phone call between two people. SPI = a teacher with a dedicated cable to each student (fast, more wires). I2C = a class where each student answers to their own name over one shared line.

Wi-Fi and Bluetooth Capabilities

Wi-Fi operating modes

Station (STA)

ESP32 joins your router as a client; used for IoT, MQTT, HTTP

```
WiFi.mode(WIFI_STA)
```

Access Point (AP)

ESP32 creates its own hotspot; phone connects directly, no router

```
WiFi.softAP("ESP32-Lab")
```

STA + AP

Both at once: configuration portal while connected

```
WiFi.mode(WIFI_AP_STA)
```

ESP-NOW

Fast router-less peer-to-peer packets between ESP32s

```
#include <esp_now.h>
```

WI-FI FACTS

- IEEE 802.11 b/g/n, 2.4 GHz only (no 5 GHz)
- Up to 150 Mbps raw; practical 10-20 Mbps
- WPA2 / WPA3 security; hidden SSIDs supported

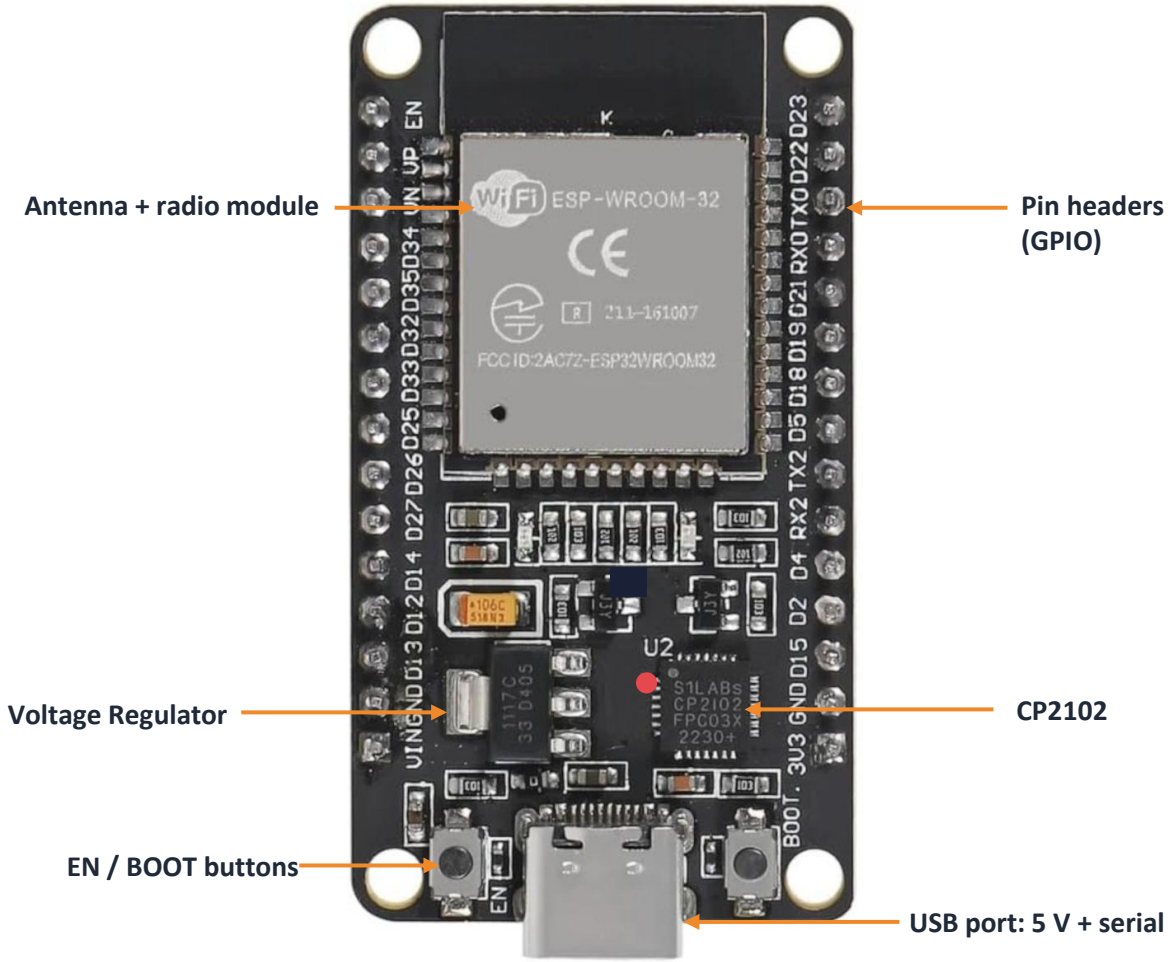
BLUETOOTH

- Classic (BR/EDR): audio, serial (SPP)
- BLE: tiny, low-power packets for beacons, sensors, phone apps
- Library: BLEDevice.h; Serial: BluetoothSerial.h

TIP

Classroom note: campus networks often block devices or use captive portals. Bring a phone hotspot or a router as a guaranteed 2.4 GHz network for the lab.

Dev Board Components and Safe Connections



DO

- Power from USB (5 V) or VIN; logic is 3.3 V
- Join all grounds (ESP32, sensor, external supply)
- Use a 220 ohm resistor in series with every LED
- Use a driver (transistor / relay board) for motors and lamps
- Use a divider or level shifter for 5 V signals

DON'T

- **Never** connect a 5 V signal directly to a GPIO pin
- Never draw more than 12 mA per pin (40 mA abs. max)
- Never use GPIO6-11 (internal flash)
- Never touch mains wiring in class: use 5-12 V loads only
- Avoid pulling strapping pins GPIO0, 2, 12, 15 during boot

Knowledge Check: Modules 1 and 2

Q1. Which statement best describes a microcontroller?

A) CPU only; RAM and I/O are external

B) CPU, memory and peripherals on a single chip

C) Needs Linux to run any program

D) Used only inside desktop computers

Q2. In the four-layer IoT architecture, sensors belong to the

A) Application layer

B) Processing layer

C) Network layer

D) Perception layer

Q3. Which ESP32 pin is INPUT ONLY?

A) GPIO2

B) GPIO21

C) GPIO34

D) GPIO25

DISCUSSION QUESTION

How would you explain to a first-year student why an ESP32 is not "a small Raspberry Pi"?

QUICK PRACTICAL TASK

Label 3V3, GND, VIN, EN, BOOT and the on-board LED pin on your board. Find two input-only pins and one DAC pin.